AI1220 • ASSIGNMENT 2 • FINAL SUBMISSION REPORT

ColabDoc

A collaborative rich-text editor with Yjs-backed realtime editing, streamed AI assistance, role-based sharing, version history, and a reviewer-friendly local setup.

REPOSITORY	FINAL BRANCH	BACKEND TESTS	FRONTEND TESTS
blackeyh/colabdoc	**main**	**66 Passing**	**20 Passing**

IMPLEMENTED SCOPE

- JWT auth with refresh flow and protected document access
- Rich-text Tiptap editor with structured JSON persistence
- Role-based sharing for owner, editor, commenter, and viewer
- Yjs-backed realtime collaboration over authenticated WebSockets
- Streamed AI suggestions with accept, edit, reject, and undo proof

FINAL STACK

- React + Vite
- FastAPI
- SQLAlchemy
- Tiptap
- Yjs
- JWT
- SQLite / PostgreSQL
- OpenAI-Compatible AI

- Version history, restore, and export to HTML/TXT

Consolidated report for the final Assignment 2 submission. Technical detail is also documented in README, DEVIATIONS, ARCHITECTURE_ADDENDUM, and DEMO_CHECKLIST.

Generated on April 19, 2026
Designed for print and PDF submission

OVERVIEW

Project Summary

ColabDoc focuses on a strong live-demo path, reliable reviewer setup, and explicit documentation of intentional scope reductions from the broader Assignment 1 design.

4
USER ROLES

2
EXPORT FORMATS

1
RUN SCRIPT

1
SUBMISSION REHEARSAL TEST

Executive Summary

The final system delivers the core Assignment 2 goals in a cohesive, test-backed implementation: collaborative editing, AI assistance, access control, versioning, export, and reviewer-friendly setup.

The repo also closes the biggest earlier architecture gap by moving live collaboration from whole-document overwrite behavior to a Yjs-backed CRDT flow while keeping a simple FastAPI deployment story.

What Changed Most Since the Earlier PoC

The final submission adds streamed AI, stronger AI auditing, export, remote cursor rendering, Yjs-based collaboration, architecture reconciliation docs, a demo checklist, and a

Core Outcomes

Reviewer-friendly startup

`./start.sh` bootstraps the local environment and runs the app.

Explicit deviation reporting

Differences from Assignment 1 are documented instead of being left implicit.

Live demo readiness

The repo includes a dedicated demo flow and an automated rehearsal path.

Stronger collaboration model

Connected collaborators now merge edits through Yjs rather than last-write-wins updates.

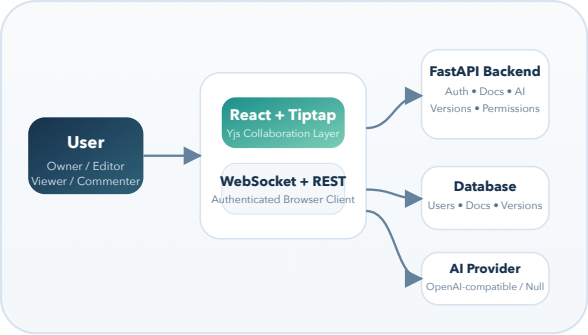
submission-style integration rehearsal.

ARCHITECTURE

Final System Design

The shipped architecture is intentionally simpler than a production multi-service platform, but it is now fully documented and internally consistent.

Architecture Snapshot



Design Decisions

Backend-managed authentication

Local JWT auth replaced the earlier external-provider plan to improve reviewer setup and testing.

Yjs in the browser

Realtime collaboration now merges connected edits through a CRDT path while keeping the server deployment lightweight.

Canonical JSON persistence

The backend still stores document JSON for export, AI context, and version history.

Provider abstraction for AI

The same backend routes support a local deterministic fallback or a real OpenAI-compatible provider.

IMPLEMENTATION

Implemented Feature Set

The final submission covers the visible rubric items a grader would expect to see in the app and in the live demo.

Authentication & Access

Registration, login, JWT bearer auth, refresh flow, and protected document access are implemented server-side.

Document Management

Create, rename, load, edit, delete, and persist rich-text documents with a compatibility layer for legacy content.

Sharing & Roles

`owner`, `editor`, `commenter`, and `viewer` roles are enforced in both the UI and backend endpoints.

Realtime Collaboration

Yjs-backed collaboration includes live edits, presence, typing indicators, cursor carets, selection highlights, and reconnect replay.

AI Assistant

Streamed AI responses, cancellation, compare-before-apply, accept/edit/reject, history, and undo after acceptance are supported.

Versions & Restore

Version snapshots can be created, listed, and restored, with reset propagation to connected collaborators.

Export

Documents export cleanly to styled HTML and readable plain text while staying deterministic and simple for grading.

Reviewer Setup

One-command startup, `.env.example`, generated local secret, API docs, and a dedicated demo checklist reduce review friction.

DOCUMENTATION

Deviation Handling and Testing

A major goal of the final submission was to remove silent mismatch between the Assignment 1 design and the delivered codebase.

Key Deviations from Assignment 1

AREA	FINAL CHOICE	REASON
Auth Provider	Local JWT auth	Simpler setup, easier review, easier testing
Repo Layout	Single repo	Cleaner submission and local bootstrap path
Realtime Backend	FastAPI relay + Yjs snapshots	Better collaboration without a heavy multi-service stack
External Services	Email/storage omitted	Not required for the Assignment 2 demo scope
Export Format	HTML / TXT	Portable, lightweight, deterministic
Local Runtime	SQLite default	Zero-config reviewer path

Testing and Evidence

66
BACKEND TESTS

20
FRONTEND TESTS

Backend coverage

Auth, documents, permissions, versions, AI, WebSockets, and submission-style rehearsal.

Frontend coverage

Login, editor bar, AI panels, cursor rendering, collaboration merge behavior, and undo proof for AI acceptance.

Submission rehearsal

A dedicated integration test walks through the same flow a grader is likely to follow.

PRESENTATION

Demo Readiness and Remaining Trade-Offs

The final repo includes a dedicated live-demo path and explicitly documents the production-grade concerns that remain out of scope.

Recommended Demo Flow

1. **Auth and protected routes**
   Log in as the owner and show that the dashboard is protected.
2. **Create and edit a document**
   Add headings, paragraphs, lists, and code to show structured rich-text editing.
3. **Share with multiple roles**
   Show `editor`, `viewer`, and `commenter` behavior in

Documented Trade-Offs

Auth Storage

Tokens remain in `localStorage` for simplicity and WebSocket compatibility.

Realtime Persistence

The latest Yjs room snapshot is kept in memory rather than a dedicated multi-instance backend.

AI Provider

The default local config uses a deterministi provider unless a real OpenAI-compatible model is configured.

Conclusion

The final Assignment 2 submission delivers a broad, well-documented, and test-backed collaborative editor. It adds real-time Yjs-based collaboration, streamed AI, export, version restore, stronger setup ergonomics, and explicit architecture reconciliation so the final repository is easier to grade and defend.

README.md | DEVIATIONS.md | ARCHITECTURE_ADDENDUM.md

separate windows.

4 **Realtime collaboration**

Type live, move the cursor, show remote selections, and mention the Yjs-backed merge model.

5 **AI streaming and undo**

Generate a streamed suggestion, accept it, and immediately undo it from the editor toolbar.

6 **Versions and export**

Restore a previous version and export the document as HTML and plain text.

DEMO_CHECKLIST.md